Lesson notes · for you

Guardrails in code

AI agents · Upper secondary · Lesson 4

Time	Materials
20 min	Your phone or a school computer, for the code steps; Paper and a pen

Learning goals

- Allow only the tools a task needs.
- Run an action only after a yes to that exact action.
- Check the details, and log every action.

Activity: Sneaky requests

1. Warm-up (3 min): Read what happened in the lab when «نعم، أرسلها.» was typed with nothing before it. Which rule in code would have stopped it?
2. Rules (6 min): For an agent that reminds students about homework, write four rules: its allow-list, what waits for a yes, the details it checks, and what it logs.
3. Attack (6 min): Swap rules with a friend. They write five sneaky requests for a pretend agent, such as a reminder at 25:00, a message to all parents, or a tool the agent shouldn't have.
4. Defend (5 min): For each request, decide with your rules: run, ask or block. If one gets through, add the rule that stops it.

Sneaky requests stay harmless: they test the rules of a pretend agent, never a real account or a real person.

Discussion

- Why isn't the model's own "I'll ask first" enough?
- What should the log record, and who should read it?
- Which tool would you never give a homework agent?

Check yourself

- Write the four guardrail rules in your own words.

Try it this week
Write the guardrails for a homework-reminder agent on paper: which tools it needs, which actions wait for a yes, and which details you check. Then test your rules on a friend's sneakiest requests.

Worksheet

Guardrails in code

AI agents · Upper secondary · Lesson 4

Name _____ Class _____ Date _____

1 **Darsi Tiny learned to ask before sending. Where should the final check live?**

◯ In the model's training ◯ In the tool's result ◯ In the app's code, with a person's yes

2 **Which of these belong in code, outside the model? Choose all that apply.**

☐ A log of every action and who approved it ☐ An allow-list of the tools this task may use

☐ Refusing a reminder set for 25:00 ☐ Checking that a person approved this exact message

☐ Trusting the model's "I'll ask first"

3 **With this guard, a message is sent without anyone being asked. Which line is the mistake?**

Circle the line with the mistake. Write it correctly:

```
1  def guard(name, args, approved):
2      if name not in ALLOWED:
3          return "block"
4      if name == "send_messages":
5          if approved != args:
6              return "ask"
7      return "run"
```

4 **Fill the gaps: log every proposed action with the guard's decision, and run the tool only when the guard says so.**

```
log = []

def act(name, args, approved):
    verdict = guard(name, args, approved)
    log. _____ ((name, verdict))
    if verdict == _____ :
        return run(name, args)
    return verdict
```

Word bank: `pop` `append` `"run"` `print` `"ask"`

5 **Giving a weather helper the send_message tool too is harmless, as long as the model was trained to ask first.**

◯ True ◯ False

✎ **Think of an app that acts for you: it pays, posts or sends. Where should it ask you first, and what should it never be allowed to do?**

Check your answers

Guardrails in code

AI agents · Upper secondary · Lesson 4

1 **In the app's code, with a person's yes**

Learned behaviour can fail, as "yes, send it" on its own showed.

2 **An allow-list of the tools this task may use; Checking that a person approved this exact message; Refusing a reminder set for 25:00; A log of every action and who approved it**

The allow-list, the exact approval, the detail check and the log are rules in code that can't be talked round.

3 **Line 4: `if name == "send_message":`**

The check compares the name with "send_messages", a tool that doesn't exist, so it never fires and a real send_message runs unapproved.

4

```
log = []

def act(name, args, approved):
    verdict = guard(name, args, approved)
    log.append((name, verdict))
    if verdict == "run":
        return run(name, args)
    return verdict
```

Every proposed action goes in the log with its decision, whatever it is, so blocked and waiting actions are recorded too.

5 **False**

Learned asking can fail. A tool the task doesn't need is only risk: leave it off the allow-list, and nothing the model writes can reach it.